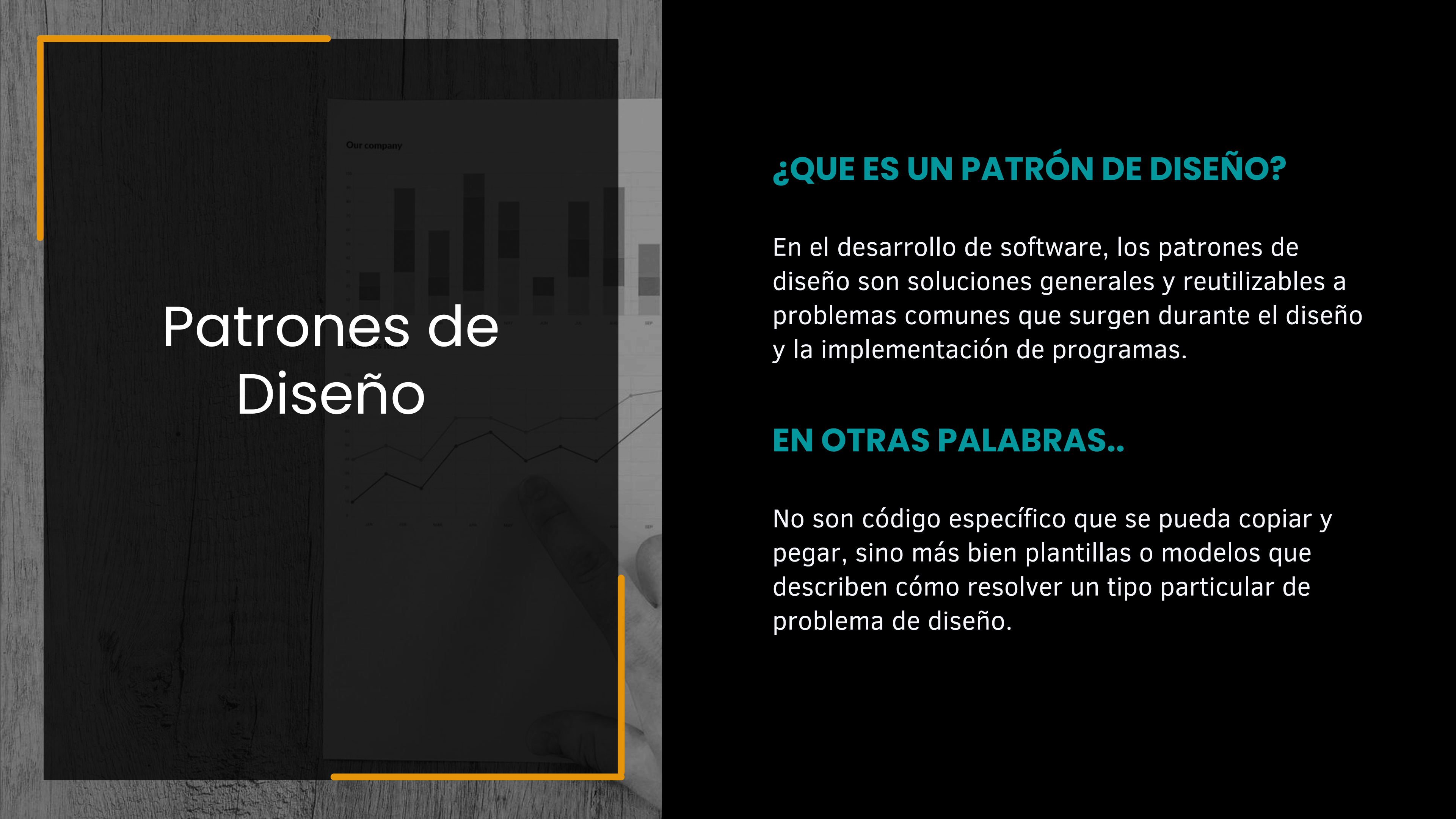


PATRONES DE DISEÑO Y PRINCIPIOS SOLID EN JAVA

PROGRAMACIÓN 3
PROF. MANGO EDUARDO

CONTENIDO

- | | |
|----|-------------------------------------|
| 01 | Patrones de diseño y su importancia |
| 02 | Factory Method |
| 03 | Singleton |
| 04 | DAO y DTO |
| 05 | Principios SOLID |
| 06 | Patrón MVC |

The background of the slide is a dark, textured surface, possibly wood. Overlaid on this is a semi-transparent image of a hand holding a tablet. The tablet screen displays two charts: a stacked bar chart at the top labeled 'Our company' and a line chart at the bottom labeled 'Business Revenue'. The text 'Patrones de Diseño' is written in large, white, sans-serif font across the middle of the slide.

Patrones de Diseño

¿QUE ES UN PATRÓN DE DISEÑO?

En el desarrollo de software, los patrones de diseño son soluciones generales y reutilizables a problemas comunes que surgen durante el diseño y la implementación de programas.

EN OTRAS PALABRAS..

No son código específico que se pueda copiar y pegar, sino más bien plantillas o modelos que describen cómo resolver un tipo particular de problema de diseño.

Patrones de Diseño

¿PARA QUE SIRVEN?

- **Reusabilidad:** Promueven la reutilización de soluciones probadas y efectivas, lo que ahorra tiempo y esfuerzo en el desarrollo.
- **Claridad y Comunicación:** Proporcionan un vocabulario común para los desarrolladores, lo que facilita la comunicación y el entendimiento sobre la estructura y el diseño del software.
- **Mantenibilidad:** Ayudan a crear código más modular, flexible y fácil de mantener, ya que los cambios en una parte del sistema tienen menos probabilidades de afectar a otras partes.
- **Escalabilidad:** Facilitan el diseño de sistemas que pueden crecer y adaptarse a nuevas funcionalidades con el tiempo.

Factory Method

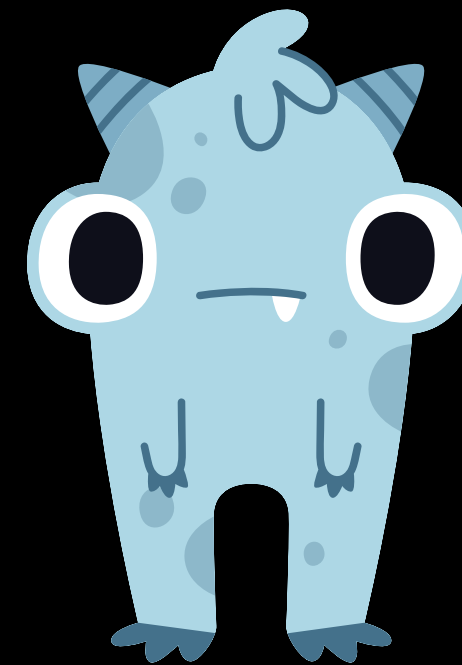
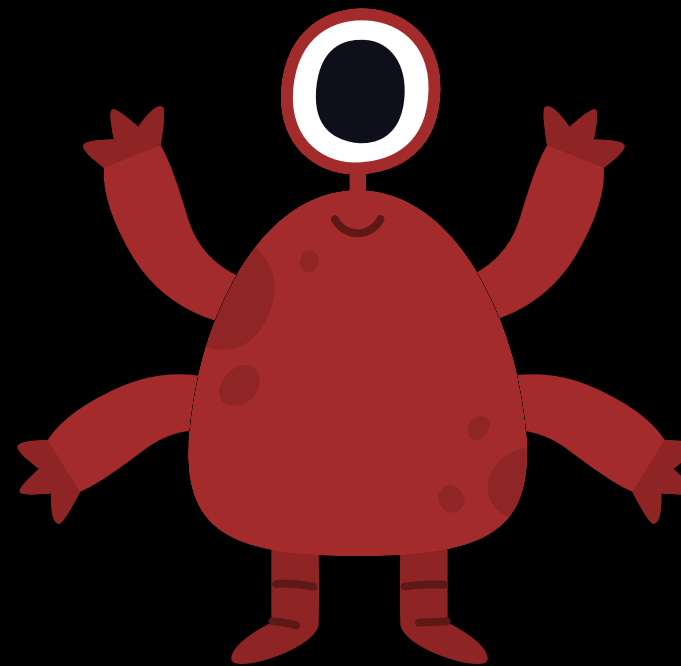
El patrón Factory Method permite crear objetos sin especificar la clase exacta de la instancia que se creará. Se usa para encapsular la lógica de creación de objetos, proporcionando flexibilidad y evitando la instanciación directa.



Imaginemos el siguiente problema

Supongamos que estamos diseñando un videojuego, donde el jugador se encontrara con diferentes tipos de enemigos. Cada tipo de enemigo tiene particularidades, pero un mismo comportamiento, ya que todos atacan y se mueven.

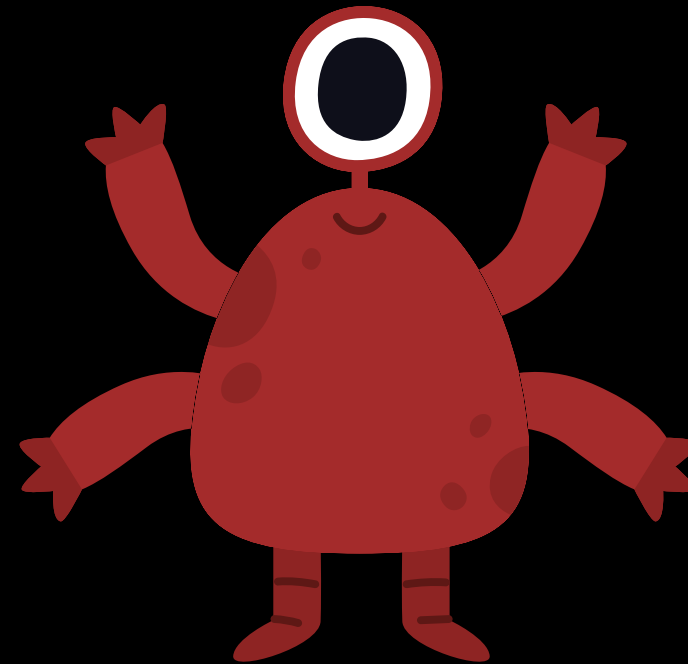
Según la abstracción ¿Como lo implementaríamos?



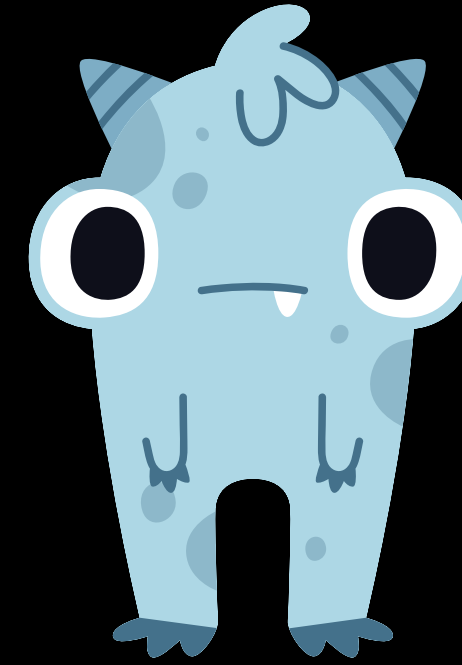
Interface <<Enemyo>>



EnemyoA
implements Enemyo



EnemyoB
implements Enemyo



EnemyoC
implements Enemyo

FactoryEnemigos

El patrón Factory Method sugiere que, en lugar de llamar al operador new para construir objetos directamente, se invoque a un método fábrica especial. No te preocupes: los objetos se siguen creando a través del operador new, pero se invocan desde el método fábrica. Los objetos devueltos por el método fábrica a menudo se denominan productos.

¿Para qué?

Cuando trabajamos con muchos tipos de objetos, el código puede volverse complicado si los creamos manualmente en muchos lugares. La clase fábrica soluciona esto: centraliza la creación de objetos, haciendo el código más organizado y fácil de mantener. ¡Además, añadir nuevos tipos de objetos es sencillo: solo modificamos la fábrica!

Singleton

El patrón Singleton garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella. Es útil para gestionar configuraciones o recursos compartidos.



La problemática

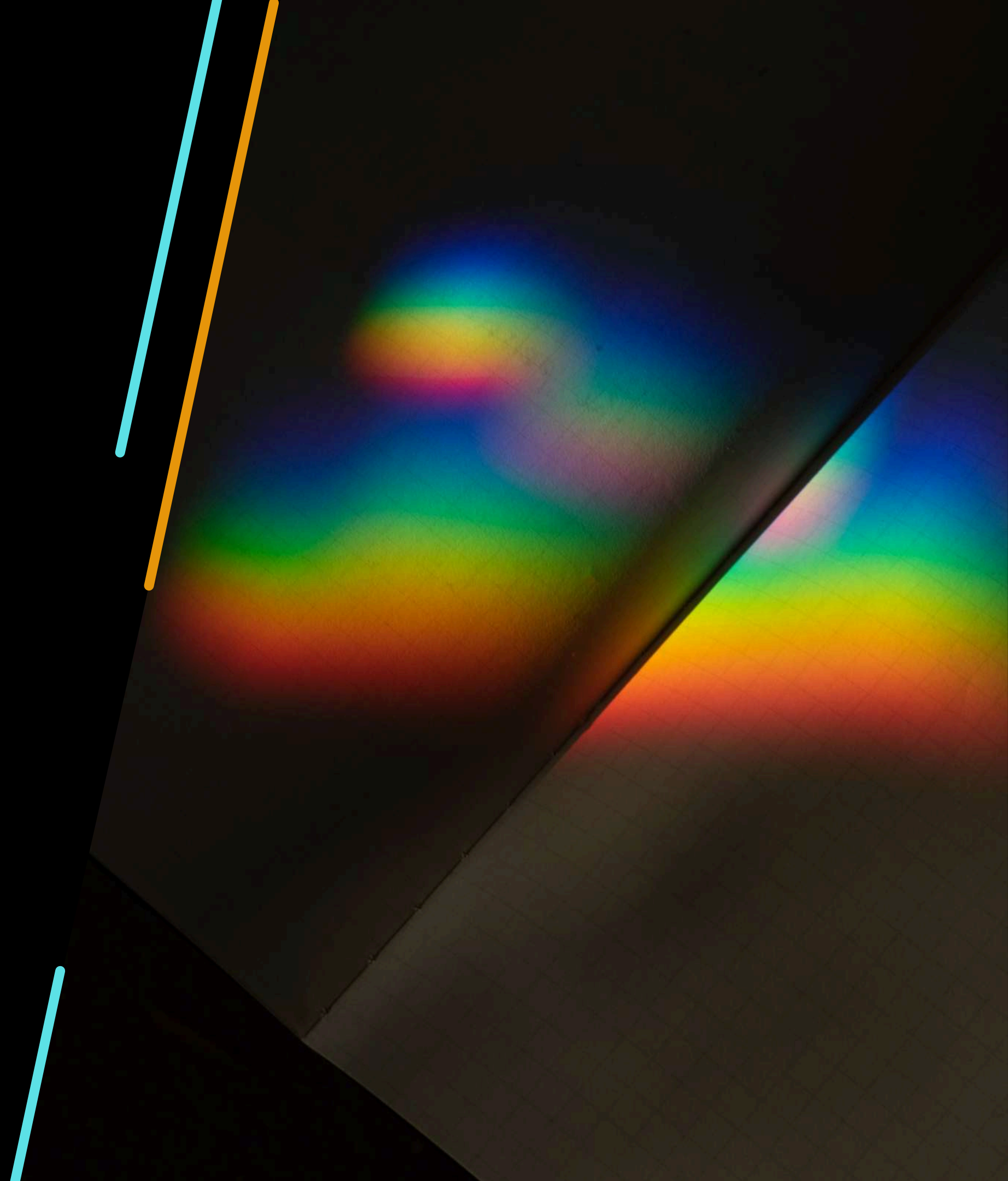
Supongamos que un mismo objeto, por ejemplo el repositorio que nos conecta a la base de datos, es necesario en multiples partes de nuestro programa.

¿Vamos a crear multiples instancias? ¿Y si lo necesito en 50 partes de mi programa? Eso significaria tener 50 conexiones a la base de datos de manera simultanea, ademas de 50 instancias de clase ocupando espacio en memoria.

Mucho mas eficiente seria tener una sola instancia, con una referencia global a esta para ser utilizada cuando se necesite.

DAO – DTO

La arquitectura DAO es un patrón de diseño que proporciona una capa de abstracción entre la lógica de negocio de una aplicación y la capa de acceso a datos. Su objetivo principal es separar las operaciones de acceso a la base de datos (u otro mecanismo de persistencia) del resto de la aplicación.



¿Para qué separar?

Digamos que tenemos una base de datos de usuarios de una billetera virtual, donde almacenamos mucha información sensible como contraseña, dirección, DNI, balance de cuentas, historial de transacciones, etc.

A traves de nuestro sistema, alguien nos pide la información de otro usuario. ¿Le daríamos todos los datos? Claramente no.

En estos casos es cuando se aplica una separación entre la entidad que se almacena en la base de datos, llamada DAO (Data Access Object u objeto de acceso de datos) y la que se transfiere, llamada DTO (Data Transfer Object u objeto de transferencia de datos). Nuestro DAO puede tener multiples DTO, quizás correspondiendo a distintos casos de uso o distintos nivel de privilegio.

Principios S.O.L.I.D

SINGLE RESPONSIBILITY PRINCIPLE (SRP)

Una clase debe tener una única responsabilidad y por ende una única razón para cambiar

OPEN-CLOSED PRINCIPLE (OCP)

El código debe estar abierto para extensión, pero cerrado para modificación.

LISKOV SUBSTITUTION PRINCIPLE (LSP)

Las subclases deben poder reemplazar a la clase base sin alterar el funcionamiento del programa

INTERFACE SEGREGATION PRINCIPLE (ISP)

Las interfaces deben ser específicas y no obligar a implementar métodos innecesarios

DEPENDENCY INVERSION PRINCIPLE (DIP)

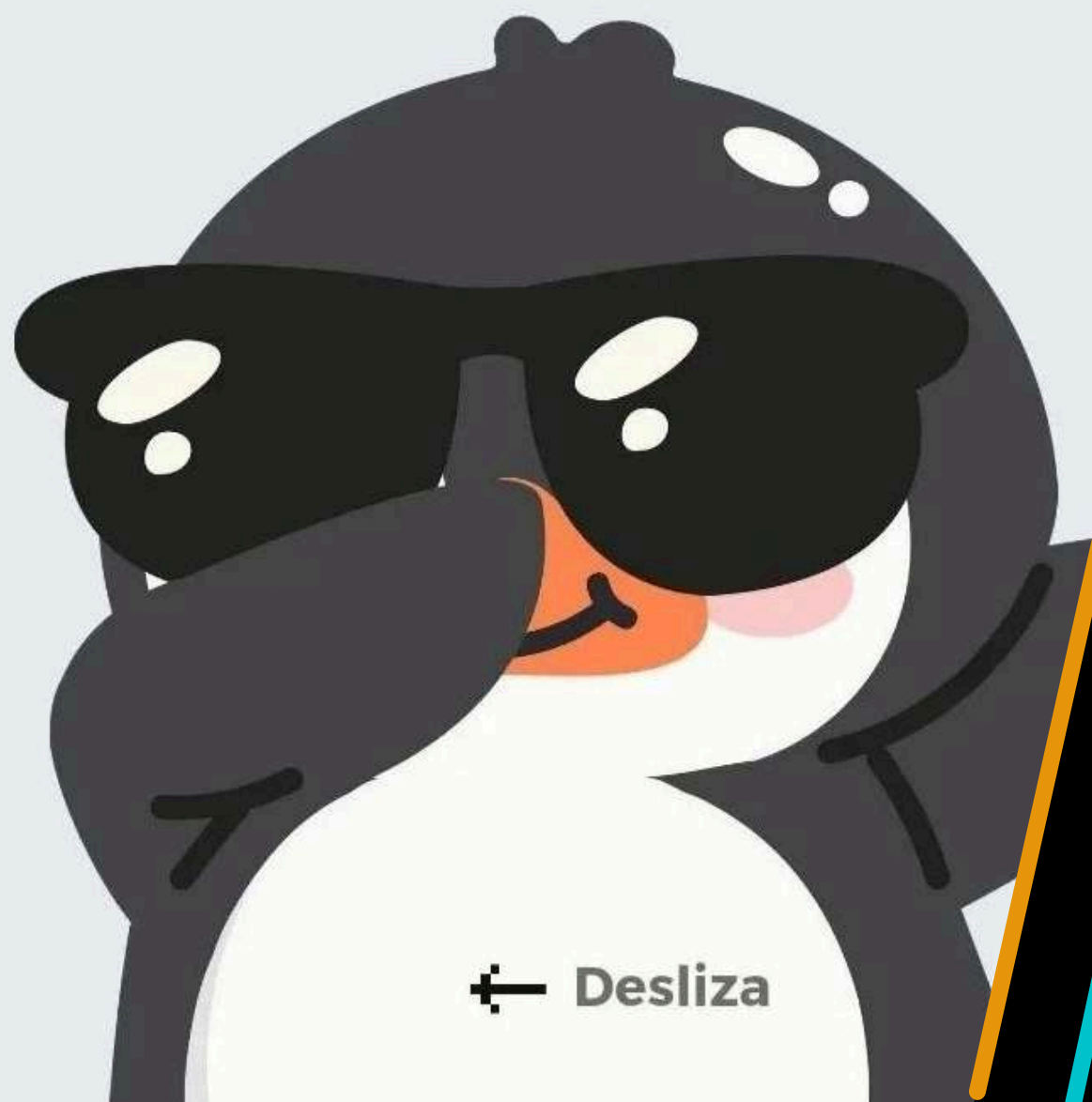
Las subclases deben poder reemplazar a la clase base sin alterar el funcionamiento del programa

Explicado con pingüinos



LOS PRINCIPIOS SOLID

EXPLICADOS CON PINGÜINO



← Desliza

S

Responsabilidad única

Cada clase debería tener una sola responsabilidad

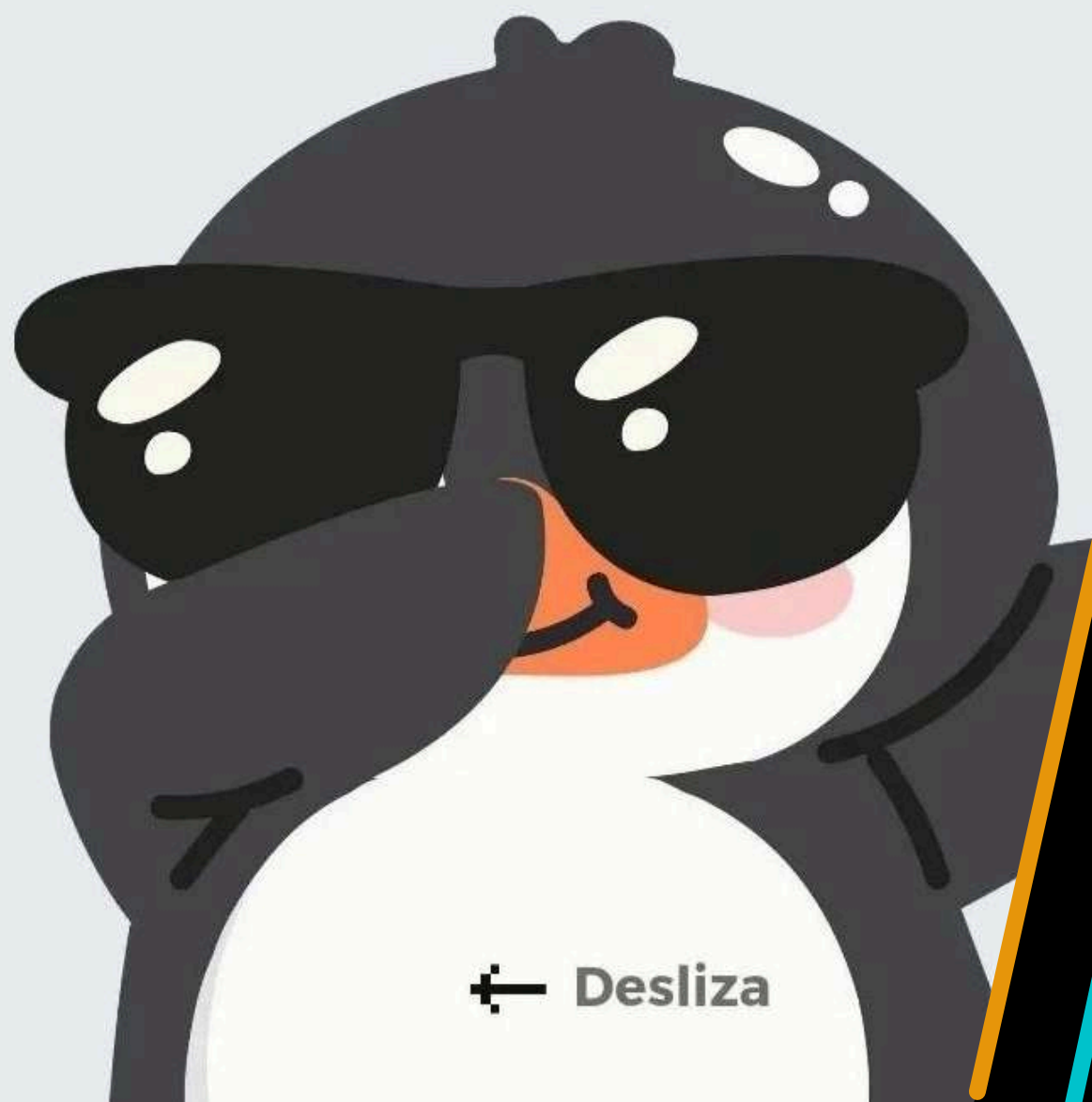
YO PESCO PECES Y
CONSTRUYO EL NIDO

YO SOLO
PESCO PECES

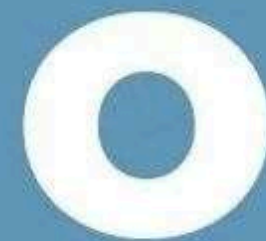


LOS PRINCIPIOS SOLID

EXPLICADOS CON PINGÜINO



← Desliza



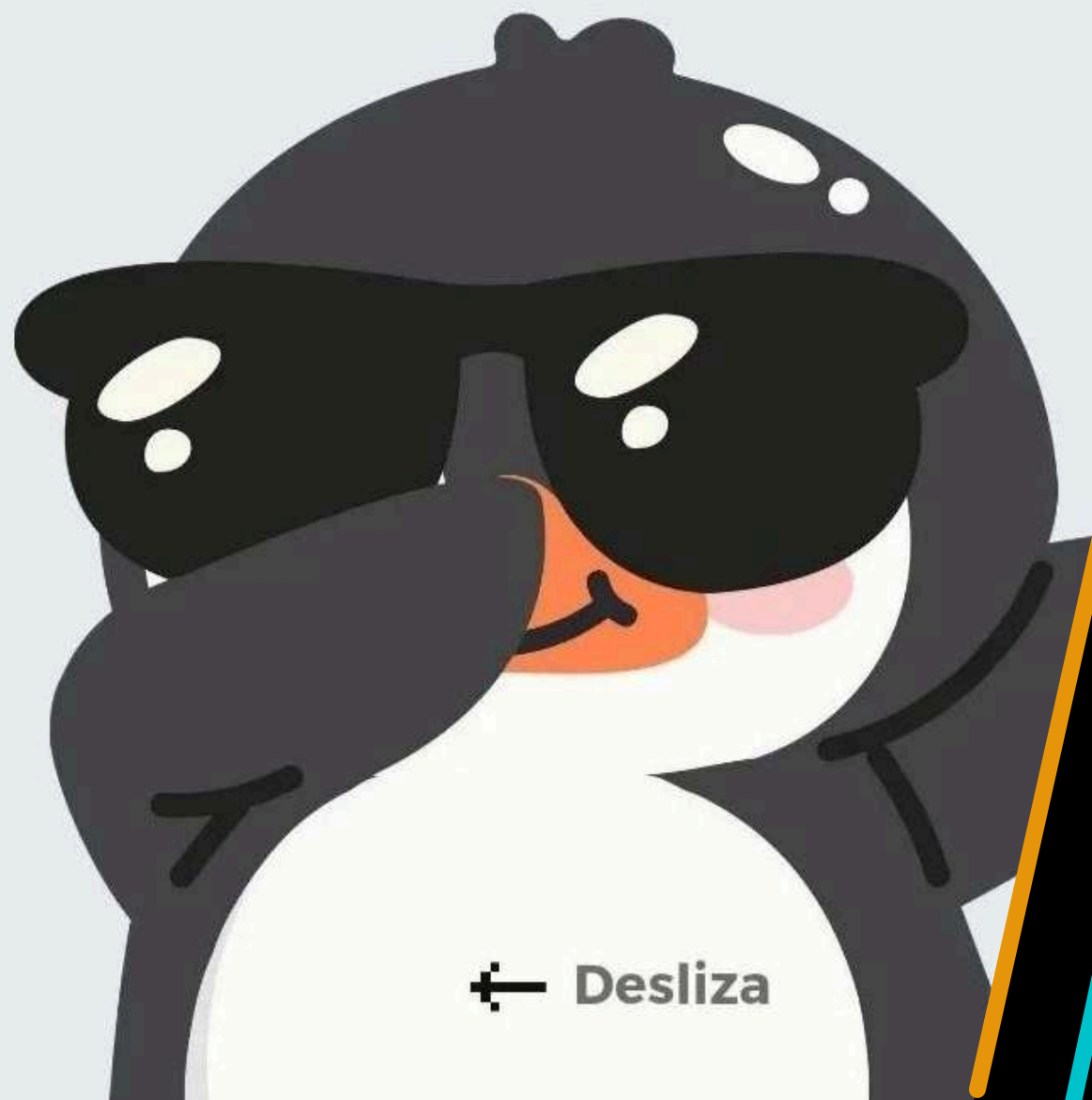
Abierto-Cerrado

Una clase debe de estar abierta a la extensión, pero cerrada a la modificación



LOS PRINCIPIOS SOLID

EXPLICADOS CON PINGÜINO



← Desliza

L

Sustitución de Liskov

Los objetos de una subclase deben poder reemplazar a los objetos de su superclase sin alterar el programa

HOLA...
SOY TUX



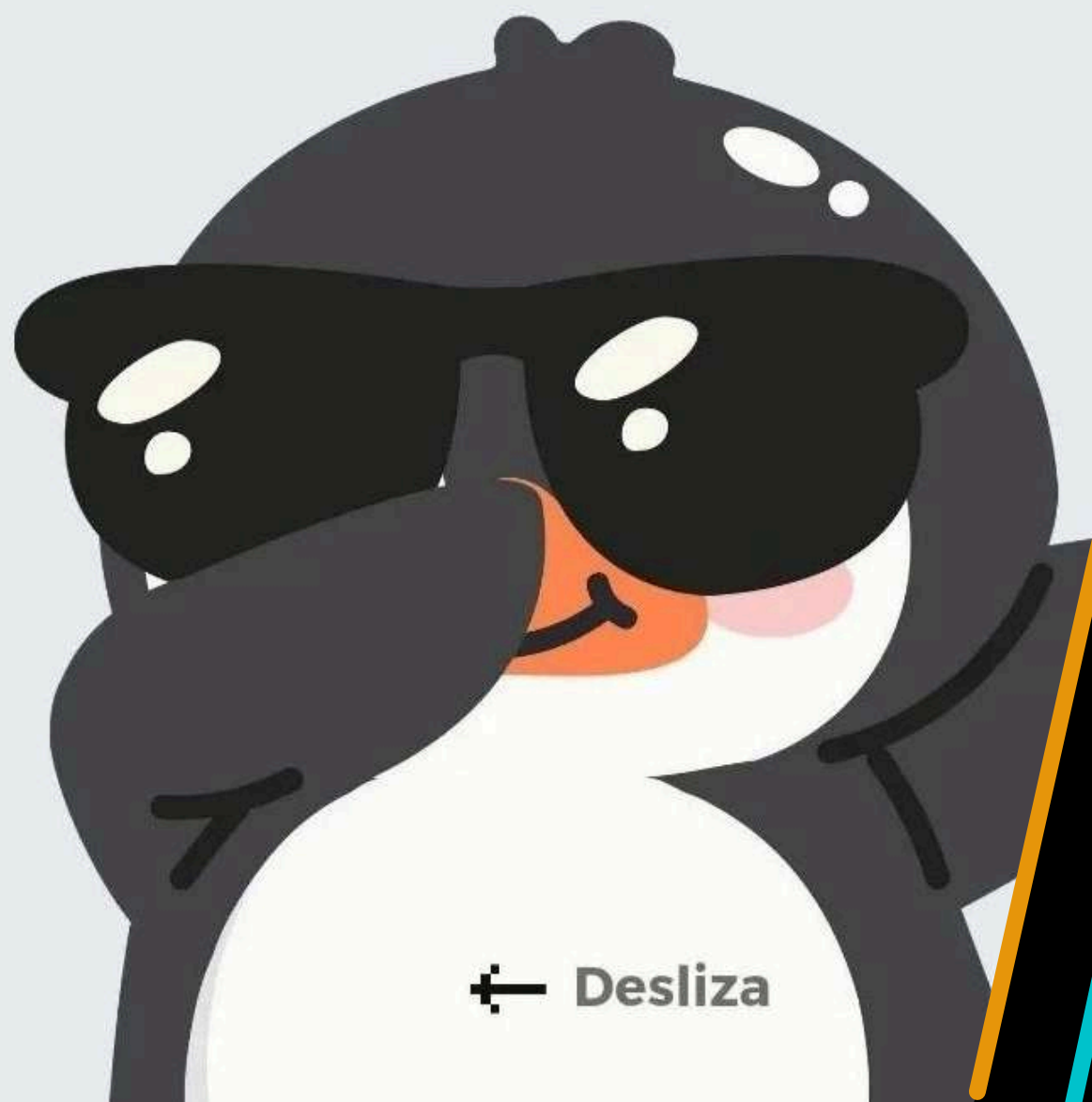
YO SOY EL PAPÀ DE TUX. PUEDO
CAMINAR Y TAMBIÉN DESLIZARME
EN EL HIELO

SI MI PAPÀ NO ESTÀ, NO
IMPORTA PORQUE YO TAMBIÉN
SÉ CAMINAR Y DESLIZARME



LOS PRINCIPIOS SOLID

EXPLICADOS CON PINGÜINO



← Desliza

Segregación de interfaces

Evita las interfaces grandes, pues pueden forzar la implementación de métodos innecesarios



LOS PRINCIPIOS SOLID

EXPLICADOS CON PINGÜINO



← Desliza

D

Inversión de dependencias

Los módulos de alto nivel no deben depender de los de bajo nivel, sino que ambos deben depender de abstracciones

PUEDO PESCAR SI
ES CON UNA DE
ESTAS REDES

YO SE UTILIZAR
LA HERRAMIENTA
QUE SEA



Patrón MVC

El patrón de diseño Model-View-Controller (MVC) es un modelo arquitectónico ampliamente utilizado en el desarrollo de software para organizar la estructura de una aplicación. Su principal objetivo es separar la lógica de negocio, la presentación y la interacción del usuario, facilitando la mantenibilidad y escalabilidad del código.



MVC

MODELO (MODEL)

El Modelo representa los datos y la lógica de negocio de la aplicación. Gestiona la información, las reglas y el estado de la aplicación. Interactúa con la base de datos para almacenar y recuperar datos.

VISTA (VIEW)

La Vista es la capa de presentación de la aplicación. Muestra la interfaz de usuario y los datos al usuario. Interactúa con el usuario para capturar sus acciones. No contiene lógica de negocio, solo se encarga de la presentación visual.

CONTROLADOR (CONTROLLER)

El Controlador actúa como intermediario entre el Modelo y la Vista. Recibe las acciones del usuario desde la Vista. Invoca al Modelo para realizar las operaciones necesarias. Selecciona la Vista adecuada para mostrar los resultados.

Ventajas

SEPARACIÓN DE RESPONSABILIDADES

REUTILIZACIÓN DE COMPONENTES

DESARROLLO PARALELO

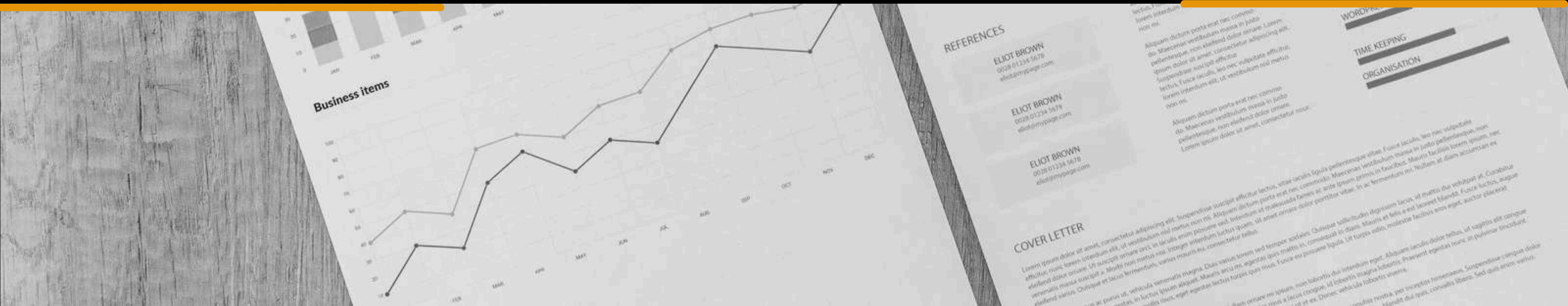
MANTENIBILIDAD

ESCALABILIDAD

FLEXIBILIDAD

Flujo de trabajo

- 1.** El usuario interactúa con la Vista (por ejemplo, a través de un formulario o un botón).
- 2.** La Vista envía la acción al Controlador
- 3.** El Controlador procesa la solicitud y, si es necesario, interactúa con el Modelo para obtener o modificar datos.
- 4.** El Modelo responde al Controlador con la información solicitada
- 5.** El Controlador actualiza la Vista con los nuevos datos



PATRONES DE DISEÑO Y PRINCIPIOS SOLID EN JAVA

PROGRAMACIÓN 3
PROF. MANGO EDUARDO



Universidad Tecnológica Nacional de Mar del Plata